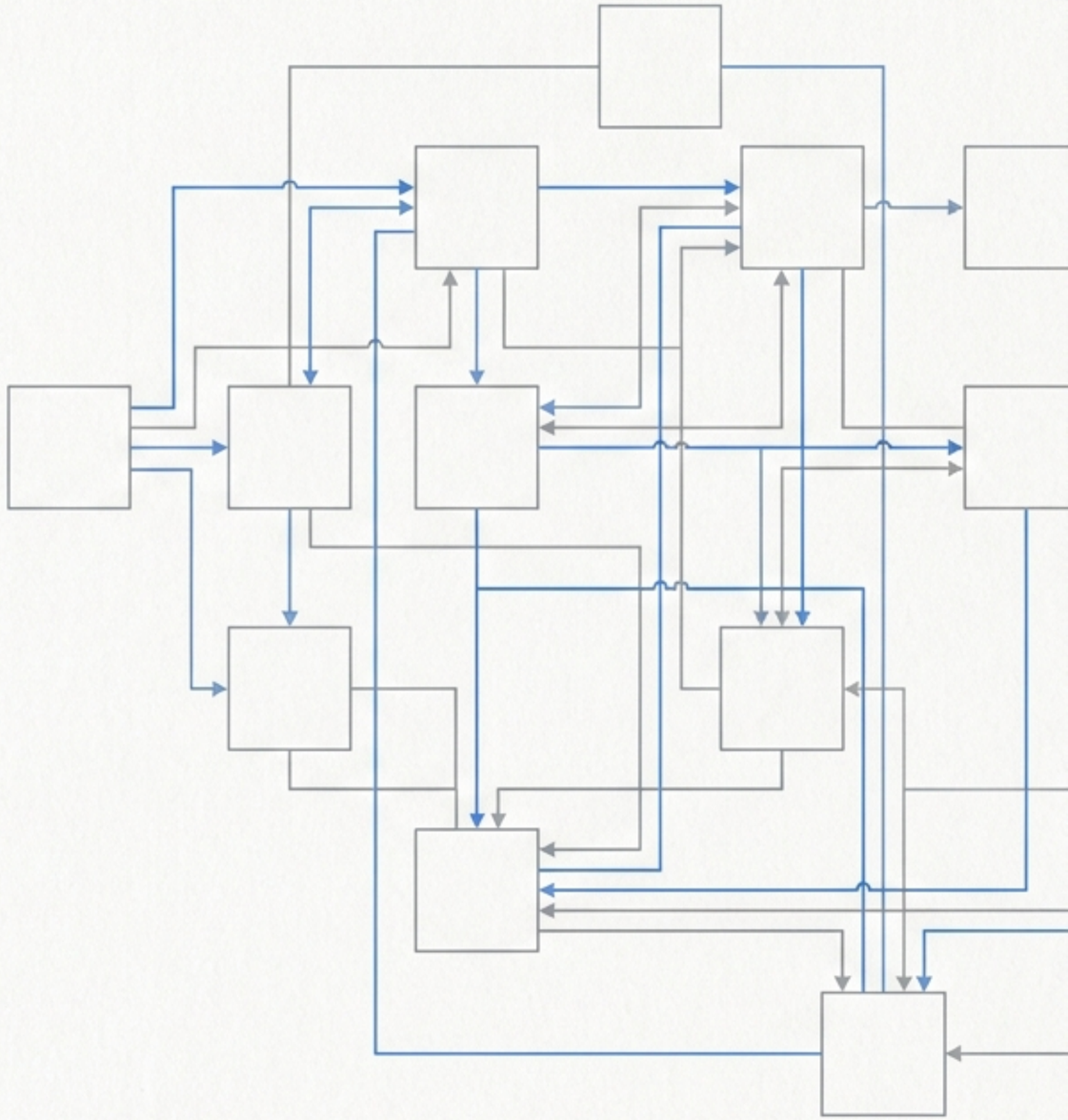
OpenStack System Architecture (Nova)

โครงสร้างสถาปัตยกรรมและ Workflow การทำงานของระบบ

> // Messaging-based, shared nothing architecture

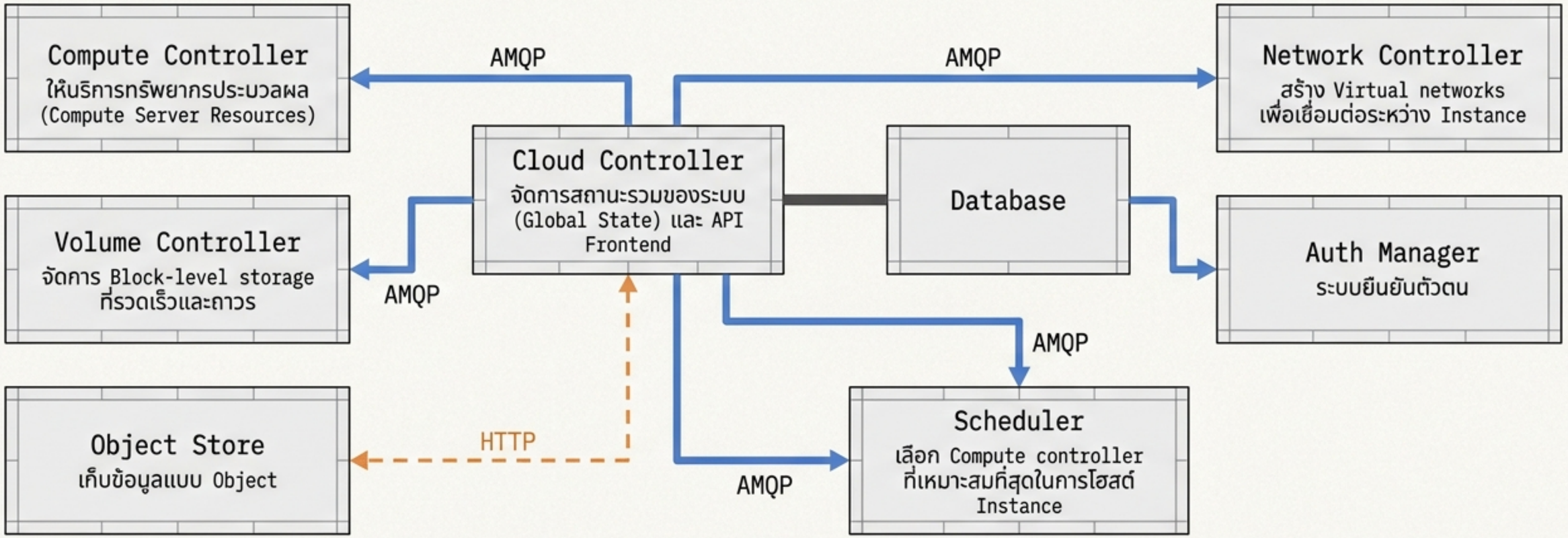


[FILE: Openstack_Architecture_part1]

Gemini Notebook

ภาพรวมสถาปัตยกรรมระบบ (Macro Architecture)

Shared Nothing Architecture: ระบบทำงานแบบกระจายศูนย์ ทุกองค์ประกอบหลักทำงานบนเซิร์ฟเวอร์หลายเครื่อง เก็บสถานะทั้งหมดไว้ใน Database กลาง



Gemini Notebook

ระบบเสมือนที่รองรับ (Hypervisor Landscape)

`>_ API ควบคุม Hypervisors รองรับการทำงานแบบ Multi-hypervisor ข้าม Availability Zones`

KVM
Kernel-based Virtual Machine

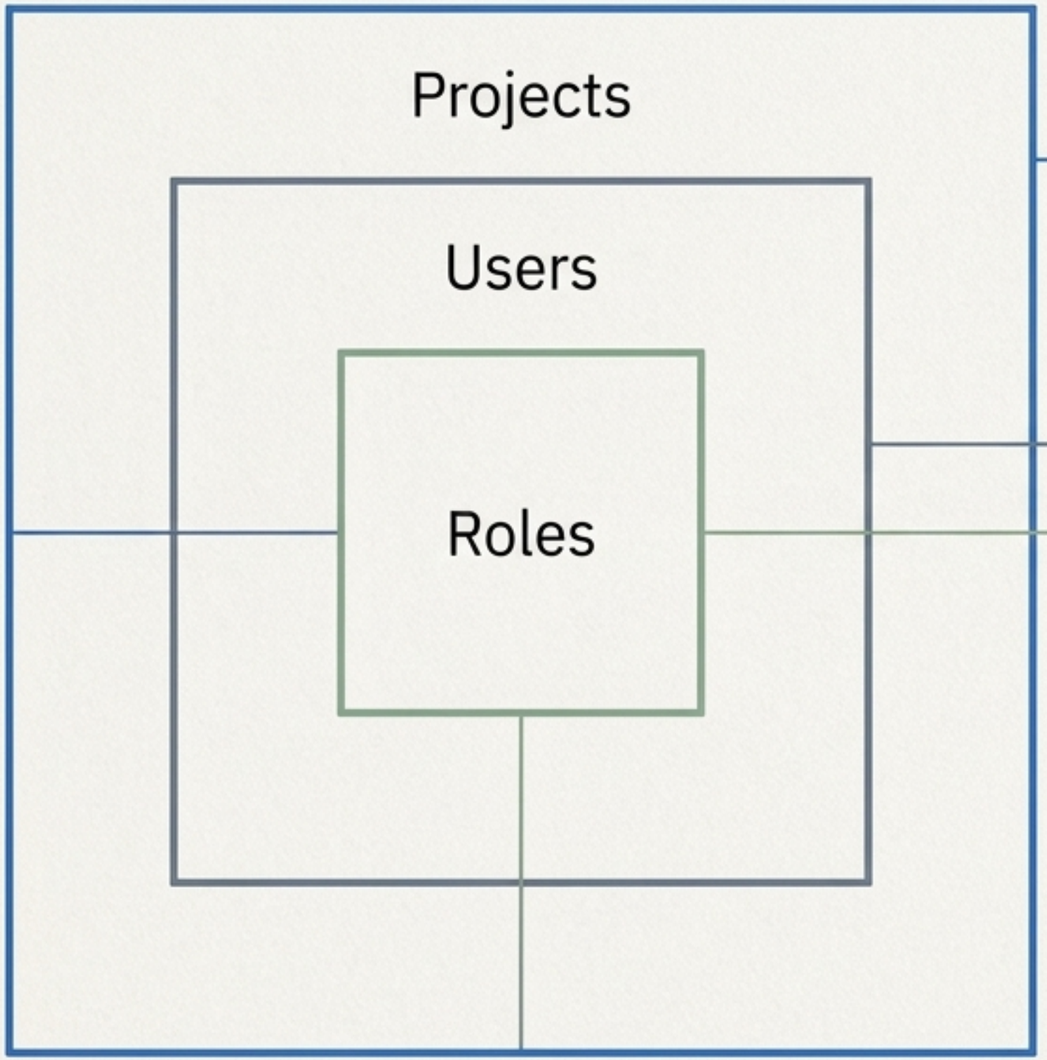
Xen
Xen

การพัฒนา OpenStack ส่วนใหญ่เน้นที่ 2 ระบบนี้เป็นหลัก

- Baremetal
- Docker
- Hyper-V
- Linux Containers (LXC)
- Quick Emulator (QEMU)
- User Mode Linux (UML)
- VMware vSphere

Gemini Notebook

โครงสร้างสิทธิ์การใช้งาน (Identity & Access Structure)



Projects (เดิมเรียกว่า **Tenants**): Container หลักสำหรับแยกทรัพยากร (เช่น VLANs, Volumes, Instances, Images, Keys) หากไม่ระบุใน API ระบบจะใช้ Project ที่มี ID ตรงกับ User

Users: ผู้ใช้งานระบบ แต่ละรายจะมี Username, Password และ Keypairs เพื่อเข้าถึง Instance

Roles: กำหนดสิทธิ์การกระทำ (Actions)

`File: policy.json`

ควบคุมสิทธิ์การใช้งาน เช่น ต้องมี Role ‘admin’ เท่านั้นจึงจะสามารถแจกจ่าย Public IP Address ได้

Gemini Notebook

การควบคุมโควตาทรัพยากร (Resource Quota Limits)

ภายในแต่ละ Project จะมีการจำกัดโควตาเพื่อควบคุมการใช้ทรัพยากรฮาร์ดแวร์:

Volumes



จำกัดจำนวน Volumes สูงสุดที่สามารถสร้างได้

Compute (CPU/RAM)



จำนวน Processor Cores และปริมาณ RAM ที่สามารถจัดสรรได้

Floating IPs



IP สาธารณะที่ใช้งานร่วมกันได้ ช่วยให้ Instance มี IP ที่เข้าถึงได้จากภายนอกแบบคงที่

Fixed IPs



IP ภายในหรือภายนอกที่ผูกติดกับ Instance นั้นๆ เมื่อเริ่มต้นระบบ

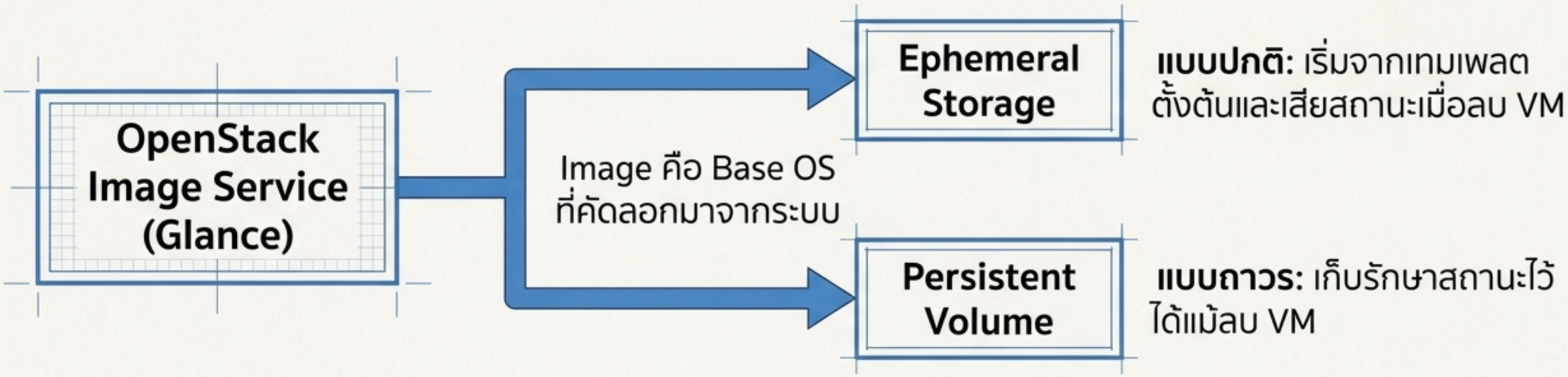
Gemini Notebook

เปรียบเทียบประเภท Block Storage

	Ephemeral Storage (พื้นที่ชั่วคราว)	Persistent Volume (พื้นที่ถาวร)
ลักษณะการคงอยู่ (Lifecycle)	ผูกติดกับวงจรชีวิตของ Instance (ลบข้อมูลทิ้งเมื่อทำลาย Instance)	เป็นอิสระจาก Instance คงอยู่แม้ปิดเครื่องหรือลบ Instance (ให้บริการโดย Cinder)
ขนาดและการจัดสรร (Size)	Root disk + Additional disk (20 - 160 GB ขึ้นอยู่กับ Flavor ยกเว้น `m1.tiny`) เป็น Raw block device	ขนาดปรับแต่งได้ตามอิสระ
File System เริ่มต้น	Ext4 (Linux), VFAT (Non-Linux), NTFS (Windows) หรือกำหนดเองผ่าน `virt_mkfs`	ขึ้นอยู่กับการฟอร์แมตของผู้ใช้
การเข้าถึง (Access & Boot)	เข้าถึงได้เฉพาะ Instance ของตัวเอง	เข้าถึงได้ทีละ 1 Instance เท่านั้น สามารถตั้งเป็น `Bootable` เพื่อสร้างระบบเสมือนแบบถาวรได้

Gemini Notebook

Building Blocks: Images (เทมเพลตระบบปฏิบัติการ)




```
> openstack image list
```

ID	Name	Status
aee1d2...	Ubuntu 14.04 cloudimg amd64	ACTIVE (พร้อมใช้งาน)
0b27ba...	Ubuntu 14.10 cloudimg amd64	ACTIVE
df8d56...	jenkins	ACTIVE

Gemini Notebook

Building Blocks: Flavors (สเปกฮาร์ดแวร์เสมือน)

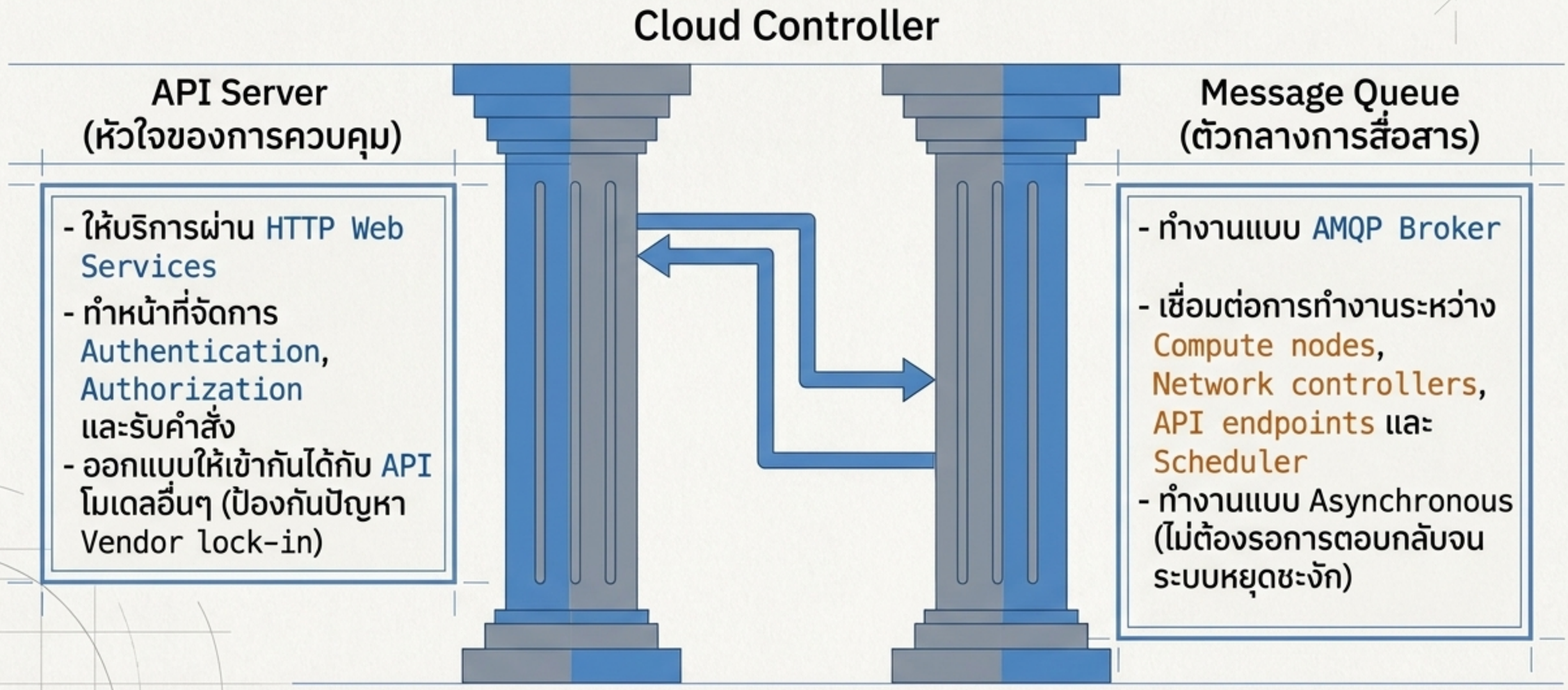
Flavors คือ Virtual hardware templates ควบคุมสเปกของ Instance
กำหนดค่าโดย Admin (แก้ไขสิทธิ์ที่ compute_extension:flavormanage ในไฟล์ policy.json)

```
> openstack flavor list
```

ID	Name	RAM	Disk	Ephemeral	VCPUs	Is_Public
1	m1.tiny	512	1	0	1	True
2	m1.small	2048	20	0	1	True
3	m1.medium	4096	40	0	2	True
4	m1.large	8192	80	0	4	True
5	m1.xlarge	16384	160	0	8	True

Gemini Notebook

โครงสร้างบริการประมวลผล (Compute Service Architecture)



Gemini Notebook

ลำดับการทำงานของระบบ (Lifecycle of a Request)

1

Request (รับคำสั่ง): API Server รับคำสั่ง HTTP จาก User

2

Authenticate (ตรวจสอบ): ตรวจสอบสิทธิ์ผู้ใช้ (Role) และความพร้อมใช้งานของทรัพยากรเป้าหมาย

3

Queue Routing (ส่งเข้าคิว): หากผ่านการตรวจสอบ คำสั่งจะถูกส่งไปยัง Message Queue (AMQP)

4

Worker Assign (รับมอบหมายงาน): Workers ที่ฟังสัญญาณอยู่ จะรับงานไปทำ (พร้อมปรับปรุงข้อมูลใน Database)

5

Response to Queue (แจ้งผลลัพธ์): เมื่อทำงานเสร็จ Worker ส่งผลลัพธ์กลับเข้าสู่ Queue

6

Relay to User (ตอบกลับผู้ใช้): API รับผลลัพธ์จาก Queue และส่งต่อกลับไปยัง User (Callback)

Gemini Notebook

หน้าที่ของ Task Controllers

Compute Worker (จัดการ Instance บน Host)

รับคำสั่งจาก API เพื่อ:

- `run_instance` (เริ่มระบบ)
- `terminate_instance` (ลบ/ทำลาย)
- `reboot_instance` (รีบูต)
- `attach_volume` / `detach_volume` (เชื่อมต่อ/ปลดดิสก์)
- `get_console_output` (ดึงหน้าจอคอนโซล)

Network Controller (จัดการทรัพยากรเครือข่าย)

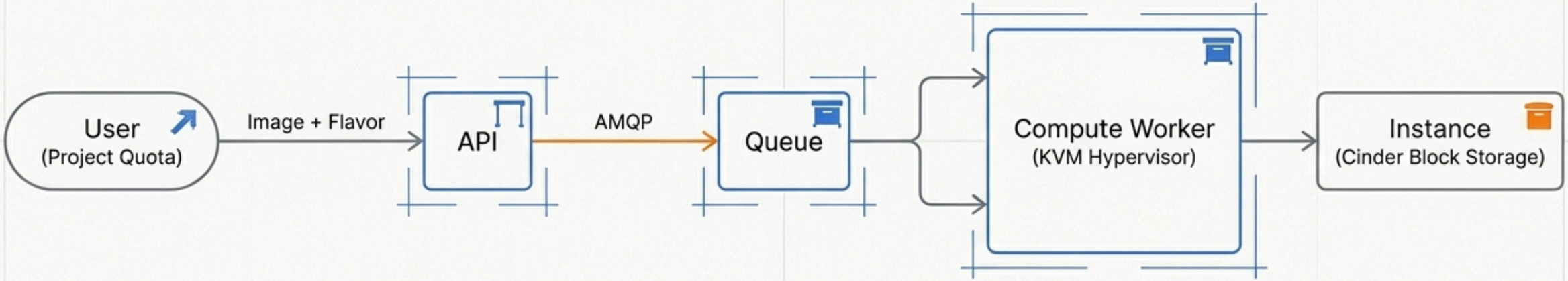
ประมวลผลคำสั่งผ่าน Message Queue เพื่อ:

- `Allocate Fixed IP addresses` (จัดสรร IP ภายใน)
- `Configure VLANs` สำหรับแต่ละ Projects
- `Configure networks` สำหรับ Compute nodes

Gemini Notebook

Architectural Synthesis (สรุปการทำงานร่วมกันของระบบ)

Everything is Connected: Asynchronous, Scalable, and Shared Nothing.



No Single Point of Failure	Resource Isolation	Flexibility
สถาปัตยกรรมไร้ศูนย์กลางอาศัย Message Queue ทำให้ขยายระบบ (Scale) ได้ไม่จำกัด	Project Quota ทำงานร่วมกับ Network Controller เพื่อแยกทรัพยากรอย่างเด็ดขาด	การแยก Image, Flavor และ Storage ออกจากกันช่วยให้สถาปัตยกรรมมีความยืดหยุ่นสูง พร้อมตอบสนองต่อทุก Request อย่างรวดเร็ว

Gemini Notebook